

Department of Computer Engineering

EC8208 : Software Architecture

Architecture Analysis and Design Report

A Microservices-Based Clinic Appointment Booking Platform

“MediConnect” : Clinic Appointment Booking System

Continuous Assessment 01

Group Take-Home Assessment

Group Members

EG/2021/4487 Dinojan V.

EG/2021/4566 Jackshan Venujan G.S.

EG/2021/4825 Tharshihan R.

EG/2021/4590 Jegan T.

EG/2021/4434 Bandara M.M.D.L.

Date of Submission: July 13, 2026

Contents

1	Introduction	2
1.1	Background of the Selected Problem	2
1.2	Problem Statement	2
1.3	Objectives of the Proposed System	2
2	Requirements Analysis	3
2.1	Functional Requirements	3
2.2	Non-Functional Requirements	4
2.3	Stakeholder Identification	5
3	Architecture Selection	5
3.1	Candidate Architecture Styles	5
3.2	Selected Style and Justification	6
4	Architectural Design	7
4.1	Context Diagram	7
4.2	Architecture Diagram	7
4.3	Component Diagram	8
4.4	Deployment Diagram	9
4.5	Class Diagram	9
4.6	Data Flow Diagram	10
5	Technology Stack Selection	10
6	Quality Attributes	11
6.1	Scalability	11
6.2	Performance	11
6.3	Security	11
6.4	Availability	12

6.5	Maintainability	12
6.6	Reliability	12
7	Conclusion	13

1. Introduction

1.1 Background of the Selected Problem

In many clinics and hospitals, patients still book appointments by phone or by walking in and waiting in long queues. There is often no easy way to see which doctor is available, at which clinic, and at what time. This leads to long waiting times, overbooking, missed appointments, and poor use of doctors' time. Most patients would prefer to arrange a visit in advance and then attend the clinic at a fixed time, instead of waiting for hours.

A web-based appointment booking platform can address this. Through a web application, a patient can find a suitable doctor and clinic, view the available time slots, and book an appointment in advance. The patient then visits the clinic in person at the booked time for the consultation. The clinic and its doctors manage their schedules, record each visit, and issue prescriptions through the same system. This keeps the booking process organised and reduces waiting, while the consultation itself remains face to face at the clinic.

1.2 Problem Statement

Manual, phone-based, or walk-in appointment handling at clinics causes long queues, double-booking, and no clear view of doctor availability. Paper-based records also make it hard to track a patient's history across visits. **There is a clear need for a secure, scalable, and highly available web platform that lets patients book appointments with clinic doctors in advance and attend in person, while giving doctors and administrators the tools to manage schedules, records, and prescriptions reliably as the number of clinics, doctors, and patients grows.**

1.3 Objectives of the Proposed System

The proposed system, **MediConnect**, focuses on a core, achievable set of objectives:

- Allow patients, doctors, and administrators to register and log in with role-based access.
- Let patients search for doctors and clinics by speciality and location, and view the available time slots.
- Let patients book, reschedule, or cancel an appointment at a physical clinic for a chosen time slot.
- After the in-clinic visit, let doctors record consultation notes and issue digital prescriptions that patients can view and download.

- Send appointment confirmations and reminders, and handle the booking or consultation fee through a secure payment step.
- Keep patient data confidential and available, and keep the system modular so that it can scale as more clinics and doctors join.

2. Requirements Analysis

2.1 Functional Requirements

Table 1: Functional requirements of MediConnect.

ID	Requirement	Description
FR-01	User Registration & Authentication	Patients, doctors, and administrators can register, verify their identity, and log in using credentials secured by token-based authentication.
FR-02	Doctor & Clinic Search	Patients can search and filter doctors by speciality, clinic, and location, and view each doctor's availability.
FR-03	Appointment Booking	Patients can view a doctor's available time slots at a clinic and book, reschedule, or cancel a physical appointment.
FR-04	Consultation Record	After the in-clinic visit, the doctor records the consultation notes and the visit outcome in the system.
FR-05	E-Prescription	Doctors can create and digitally sign prescriptions that patients can view and download.
FR-06	Electronic Health Records	The system stores and retrieves each patient's visit history, prescriptions, and consultation notes.
FR-07	Payment Processing	Patients pay the booking or consultation fee through a secure payment step and receive a receipt.
FR-08	Notifications	The system sends appointment confirmations and reminders via email and SMS.
FR-09	Rating & Feedback	Patients can rate and review doctors after a completed visit.

ID	Requirement	Description
FR-10	Administration	Administrators can verify doctor credentials, manage clinics and users, and monitor platform activity.

2.2 Non-Functional Requirements

The following non-functional requirements define the quality attributes the system must meet. They are the main drivers of the architectural decisions in this report.

Table 2: Non-functional requirements of MediConnect.

ID	Attribute	Requirement
NFR-01	Scalability	The platform shall scale horizontally to support at least 10,000 concurrent users during peak booking hours.
NFR-02	Performance	95% of API requests shall respond within 500 ms under nominal load.
NFR-03	Security	All data in transit shall use TLS 1.2+; sensitive data at rest shall be encrypted; access shall be role-based.
NFR-04	Availability	The system shall maintain 99.9% uptime, with no single point of failure for core services.
NFR-05	Reliability	Appointment, payment, and prescription operations shall be transactionally consistent and recoverable after failure.
NFR-06	Maintainability	Services shall be independently deployable, with clear interfaces, to allow isolated updates.
NFR-07	Privacy/Compliance	The platform shall comply with health-data protection principles (for example, HIPAA or GDPR style consent and audit logging).
NFR-08	Usability	The user interface shall be responsive and accessible on modern web browsers.
NFR-09	Interoperability	The system shall expose RESTful APIs and support integration with external pharmacy and laboratory systems.

2.3 Stakeholder Identification

The table below identifies the parties whose needs drive the requirements above.

Table 3: Key stakeholders and their roles.

Stakeholder	Interest / Role
Patients	Primary end users who book appointments in advance and attend the clinic in person.
Doctors	Manage their schedules, see booked appointments, and record visit notes and prescriptions.
Clinics	Provide the physical location and doctors, and manage their listed availability.
Administrators	Verify doctors, manage clinics and users, and monitor platform health and compliance.
Pharmacies & Labs	External partners fulfilling prescriptions and diagnostic orders.
Payment Provider	Third-party service that processes booking and consultation payments.
Regulators	Authorities enforcing health-data privacy and clinical compliance.
Development & Ops Team	Build, deploy, scale, and maintain the platform.

3. Architecture Selection

3.1 Candidate Architecture Styles

Several common architecture styles were evaluated against the requirements above. Because each style involves trade-offs, the options are judged by how well they serve this system's goals rather than by any single style being best in general.

Table 4: Comparison of candidate architecture styles.

Style	Strengths	Weaknesses for this domain
Monolithic / Layered (N-tier)	Simple to build and deploy; clear separation of presentation, business, and data layers; low initial complexity.	A single deployable unit scales as a whole, couples unrelated concerns, and one failure can bring down the entire system, which is a poor fit for 99.9% availability and independent scaling.
Client-Server	Well-understood request and response model; centralises data and logic.	On its own it does not address independent scaling or fault isolation of distinct functions such as booking and payment.
Event-Driven	Excellent for asynchronous, loosely coupled flows such as notifications and audit logging.	Harder to reason about end-to-end consistency, and it does not by itself structure the synchronous request and response core (booking, payment).
Microservices	Small, independent services that are separately scalable and deployable, with fault isolation, per-service technology choice, and clear team ownership.	Higher operational complexity (orchestration, distributed data, networking).

3.2 Selected Style and Justification

MediConnect adopts a Microservices architecture, supported by an event-driven message bus for asynchronous tasks, an API Gateway as the single client-facing entry point, and a layered organisation inside each service. This is a key architectural decision that shapes the technology, component structure, and integration of the whole system. The justification follows from the requirements and goals:

- **Independent scalability (NFR-01).** The appointment booking service can be scaled on its own during peak booking times, separately from lighter services such as prescriptions.
- **Fault isolation and availability (NFR-04).** A failure in the payment service must not stop patients from booking or doctors from recording a visit; service boundaries contain failures.

- **Maintainability (NFR-06).** Each service has a single responsibility and communicates only through published interfaces, which allows isolated, frequent deployments.
- **Security (NFR-03).** A central API Gateway enforces authentication, rate limiting, and TLS termination uniformly, while sensitive services sit behind it.
- **Asynchronous decoupling.** A message broker carries events (for example, AppointmentBooked) to the notification and records services without blocking the main request path.

Each service owns its own database (the database-per-service pattern), which prevents hidden coupling through a shared schema and lets each store be tuned for its workload.

4. Architectural Design

The proposed architecture is presented below through a set of standard design diagrams.

4.1 Context Diagram

The context diagram shows MediConnect as a single system and its interactions with external actors and systems.

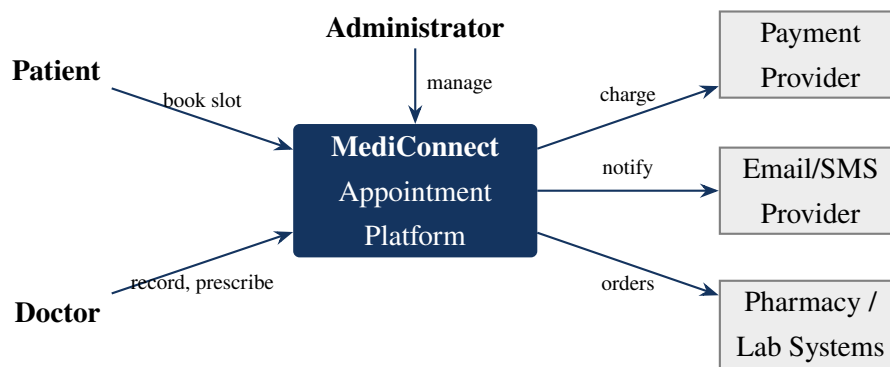


Figure 1: Context diagram: MediConnect and its external entities.

4.2 Architecture Diagram

The high-level architecture organises the system into client, gateway, service, messaging, and data tiers.

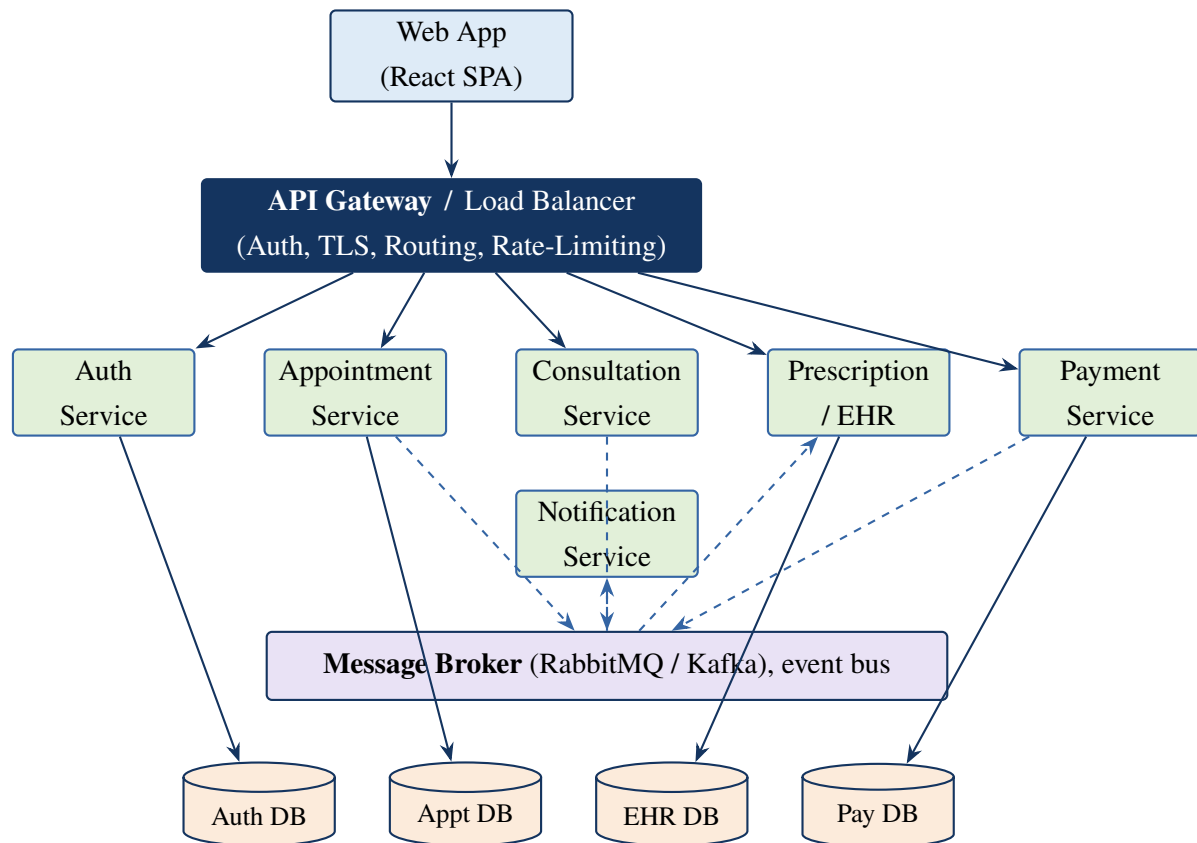


Figure 2: High-level microservices architecture (solid = synchronous REST, dashed = asynchronous events).

4.3 Component Diagram

Internally, each service follows a layered structure (controller, business logic, data access), so that upper layers do not bypass lower ones. For example, the controller never accesses the database directly. The figure details the **Appointment Service** as a representative example.

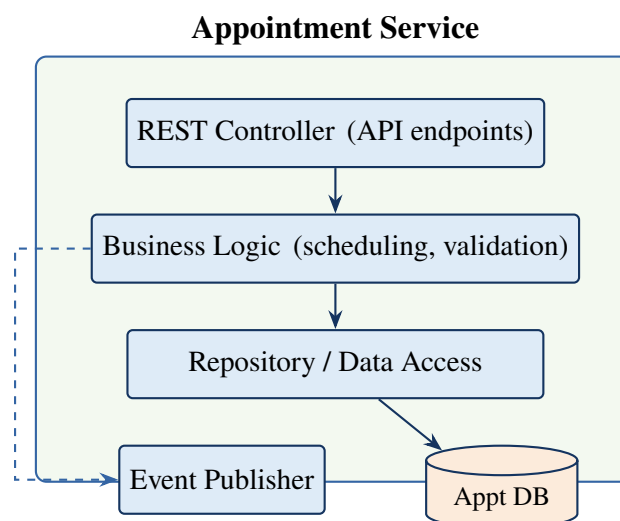


Figure 3: Component decomposition of a representative microservice (Appointment Service).

4.4 Deployment Diagram

Services are containerised and orchestrated on a Kubernetes cluster, with managed databases and a CDN for the web client.

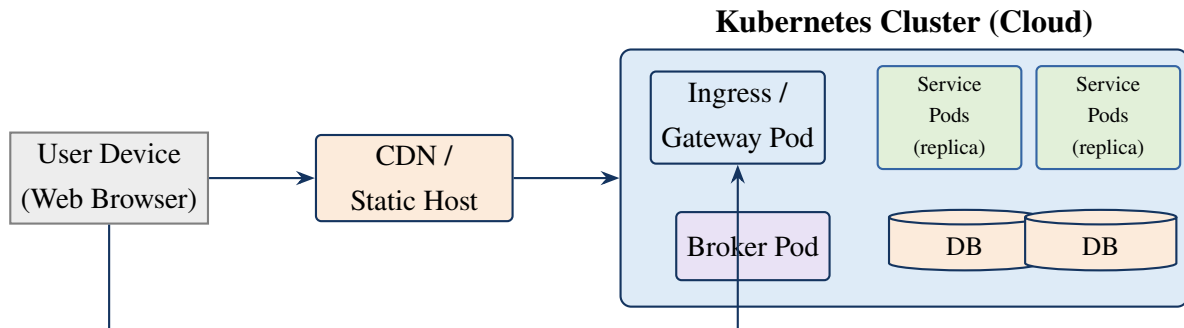


Figure 4: Deployment view: containerised services on a managed Kubernetes cluster.

4.5 Class Diagram

The core domain entities and their relationships are shown below (attributes abbreviated). An appointment is booked by a patient, is with a doctor, and takes place at a clinic.

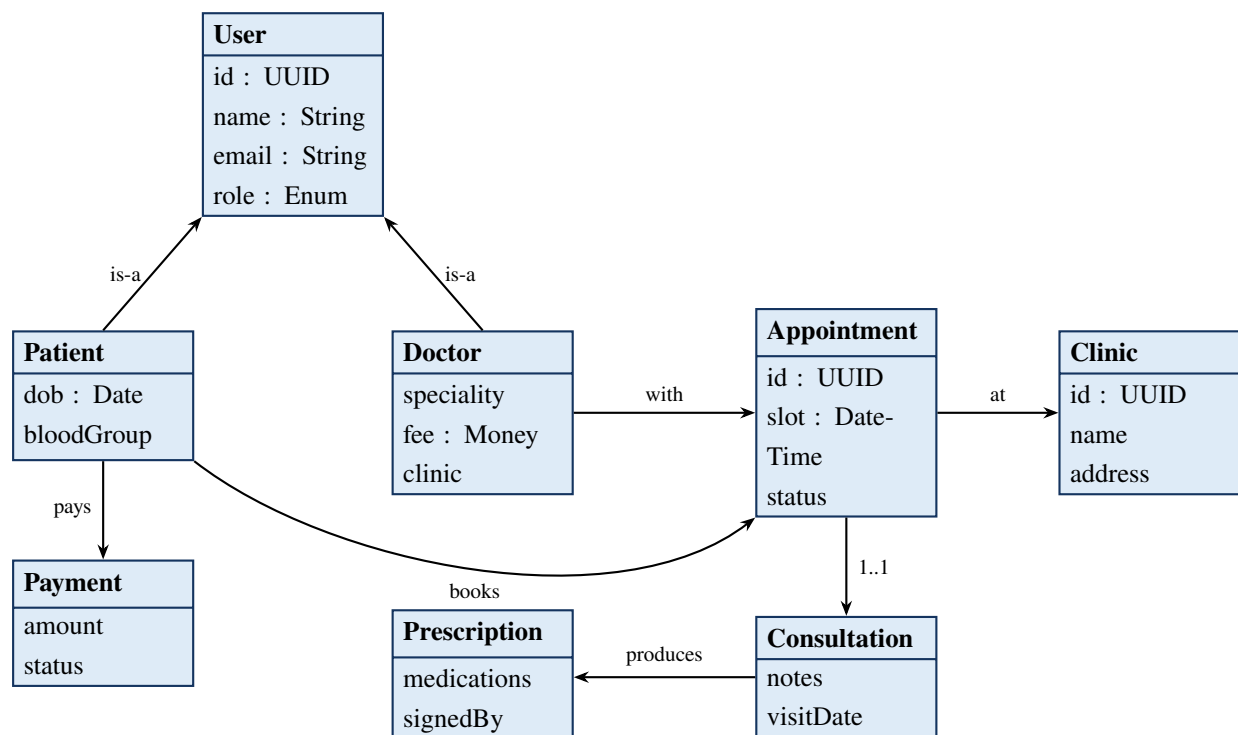


Figure 5: Domain class diagram (core entities and relationships).

4.6 Data Flow Diagram

The following data flow diagram (Level 1) traces the “search, book, and pay for an appointment” flow and shows how the components interact at run time.

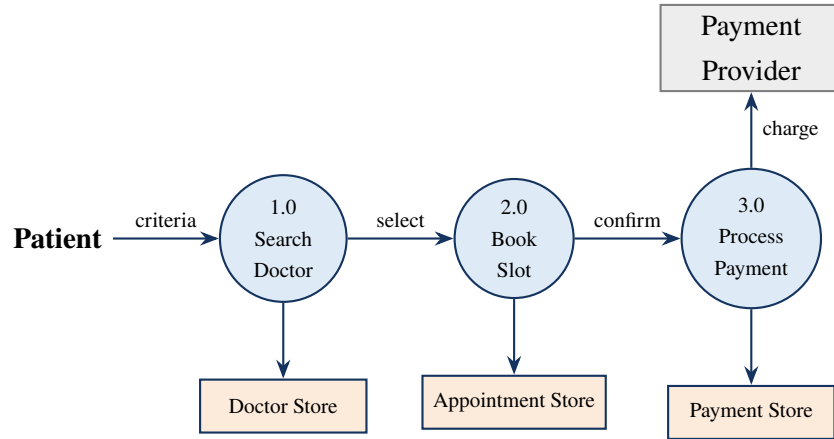


Figure 6: Level-1 data flow diagram for the search, booking, and payment process.

5. Technology Stack Selection

The stack is chosen to support the microservices style, polyglot persistence, and consistent operational tooling.

Table 5: Technology stack and rationale.

Layer	Technology	Rationale
Front-end	React.js + TypeScript	Component-based, responsive single-page web app.
API Gateway	Kong / NGINX	Central auth, routing, TLS termination, rate limiting.
Backend services	Node.js (Express) and Spring Boot	Lightweight services in Node; Spring Boot where strong typing and transactions matter.
Messaging	RabbitMQ / Apache Kafka	Asynchronous, decoupled event delivery.
Databases	PostgreSQL (transactional), MongoDB (records)	Polyglot persistence, database-per-service.
Caching	Redis	Session and token cache, doctor and slot availability cache.

Layer	Technology	Rationale
Authentication	JWT + OAuth 2.0	Stateless, role-based access tokens.
Containerisation	Docker + Kubernetes	Packaging, orchestration, auto-scaling, self-healing.
CI/CD	GitHub Actions / Jenkins	Automated build, test, and deploy pipelines.
Monitoring	Prometheus + Grafana, ELK	Metrics, dashboards, centralised logging.
Cloud	AWS / Azure	Managed Kubernetes, databases, and CDN.

6. Quality Attributes

This section explains how the design addresses each required quality attribute.

6.1 Scalability

Each microservice is stateless behind the gateway and deployed as multiple replicas. Kubernetes' Horizontal Pod Autoscaler adds or removes pods based on CPU and latency, so the appointment booking service can scale on its own during peak times, separately from lighter services. Redis caching and the database-per-service pattern remove shared bottlenecks, satisfying NFR-01.

6.2 Performance

The API Gateway routes efficiently and offloads TLS, and Redis caches frequent reads such as doctor listings and slot availability, which keeps the median request below the 500 ms target. Database indexing on doctor, clinic, and slot fields keeps search and booking queries fast (NFR-02).

6.3 Security

Security is applied in layers (defence in depth): TLS 1.2+ for all traffic, JWT and OAuth 2.0 with role-based access control at the gateway, encryption at rest for records and payment data, input validation in each service, and audit logging. The payment service is isolated and never stores raw card data, delegating to a secure payment provider (NFR-03, NFR-07).

6.4 Availability

Replicated, stateless services and Kubernetes self-healing remove single points of failure. The message broker decouples services, so a temporary outage of the notification service does not block bookings; events are queued and processed when it recovers. Multi-zone deployment supports the 99.9% uptime goal (NFR-04).

6.5 Maintainability

Clear service boundaries, well-defined interfaces, and independent CI/CD pipelines let teams change and deploy one service without redeploying the whole system. Containerisation gives consistent environments from development to production (NFR-06).

6.6 Reliability

Critical operations such as booking, payment, and prescription use transactional databases and the Saga pattern for consistency across services, with compensating actions on failure. Retries with idempotency keys and durable message queues ensure that events are not lost, and health checks restart failed services automatically (NFR-05).

Table 6: Mapping of quality attributes to architectural tactics.

Quality Attribute	Architectural Tactic	Realised By
Scalability	Horizontal replication	K8s HPA, stateless services, Redis
Performance	Caching, edge routing	Redis, API Gateway, database indexing
Security	Defence in depth	TLS, JWT/OAuth, RBAC, encryption, audit logs
Availability	Redundancy and isolation	Replicas, self-healing, message broker
Maintainability	Modularity	Microservices, clear interfaces, CI/CD
Reliability	Consistency and recovery	Saga pattern, idempotency, health checks

7. Conclusion

This report analysed the problem of manual, queue-based appointment handling at clinics and proposed **MediConnect**, a web platform that lets patients book appointments with clinic doctors in advance and then attend in person. The system is built on a microservices architecture with an API Gateway, an event-driven message bus, and a layered organisation inside each service. The requirements analysis identified the functional and non-functional requirements and the stakeholders they serve, and a comparison of common architecture styles showed that microservices best satisfy this system's needs for independent scalability, fault isolation, security, and maintainability.

The design was presented through context, architecture, component, deployment, class, and data-flow diagrams, and supported by a practical technology stack. The main architectural decisions, namely the use of microservices, a central API Gateway, a database-per-service model, and an event-driven backbone, were justified against the system goals, and each quality attribute was traced to a concrete design tactic. The scope was kept focused so that the core functionalities can be built as a working prototype. The result is an architecture that is scalable, secure, available, and maintainable, and that provides a clear foundation for implementation.